

Context Health MLP Cognitive Software Factory

Project report: what was built, every run, every probe, what worked, what did not, and how to use the result
28-30 September 2026

1. Summary

The goal was a deterministic, Python-driven factory in which isolated language-model agents search for a small MLP that keeps an autonomous agent's context window healthy. The MLP watches telemetry each step and picks one of six interventions (do nothing, compact, prune tool output, re-inject instructions, reset from a checkpoint, retrieve memory). The harness, not the language models, owns the loop, the stopping rules and the model routing.

- **Built and verified:** the harness, a five-rule model/effort router with an independent audit, five isolated agents, a deterministic simulator with a 60-second limit, a dynamic state space that starts from one seed dimension, and 52 automated tests.
- **Seven factory runs** (83 iterations, 76 scored). Every completed run ended on a Python-decided stopping rule and passed all per-iteration and whole-run checks.
- **Score progress** on a scale where doing nothing is 0, a hand-written heuristic is 50 and a clairvoyant planner is 100: about 76 for the first fair baseline, about 81 on scenarios that were never used for selection for the final models.
- **Practical ceiling:** an information-ceiling probe puts the limit of this training approach near 83, so the search is within roughly two points of it.
- **Deliverable:** a deployable policy package in `workspace/deploy/` whose decisions match the simulator exactly.

2. What was built

2.1 The golden rule

The harness is deterministic; the work is creative. Agents only return JSON. Python validates every output, applies every state change, chooses every model, and decides when to stop. No agent output contains a field that can end the loop.

2.2 Components (all under factory/)

Component	File	Role
Orchestrator	<code>run_factory.py</code>	State machine. Runs iterations in synchronous waves, retries up to three times, poisons a coordinate after three failures, halts on the stopping rules.
Router	<code>router.py</code> , <code>router_audit.py</code>	Escalation ladder: Sonnet 5 standard, Sonnet 5 high, Opus 5.5 standard, Opus 5.5 high. Fable 5.1 only for Markdown formatting. Every decision is re-derived by a second implementation and logged.
Agents	<code>agents.py</code> , <code>prompts/</code>	Meta (mutates the search space, proposes coordinates), Worker (writes <code>model.py</code>), Judge (scores), Doc (writes the paper). Each call has no tools and sees only the prompt Python built.
Isolation	<code>isolation.py</code>	Per-agent payload whitelists, per-experiment canary strings, and a guard that scans every prompt for another experiment's id. A hit aborts the factory.

Component	File	Role
State space	state_space.py	Generic mutation engine (add/prune dimensions and values). Contains no dimension names.
Stopping rules	done_checker.py	No new dimension in 10 iterations; best judge score of the last 10 iterations not more than 2.0 above the earlier best; iteration limit.
Simulator	simulator/	Pure Python. Trains the Worker's model, runs it closed-loop, writes a log with exactly 50 turns. Killed after 60 seconds.
Verification	verify_run.py, tests/	Checks every Definition-of-Done bullet from artefacts on disk; 52 tests including full runs with a scripted mock LLM.

2.3 The simulated environment

Each scenario is a 40-step agent session with hidden dynamics: context utilisation, redundant and stale content, instruction salience, fact integrity, error loops, topic shifts and overflow. There are seven scenario families with per-scenario parameter jitter. The policy sees 21 noisy telemetry fields per step. Reward is +1 per successful sub-task, minus intervention cost, minus penalties for recall misses and overflow, plus 10 if at least 24 of 40 sub-tasks succeed.

The score (sim_score) is piecewise linear in mean return: do-nothing policy 0, a hand-written threshold heuristic 50, a clairvoyant search over the true future 100. The clairvoyant bound is unreachable by any policy that only sees observations.

3. Every run

Run	Iterations (scored / poisoned)	Best judge	Best sim_score	LLM calls	Cost (USD)	Outcome
v4.0	6 / 0	77	76.78	30	3.38	sequential; stopped by me to add parallelism
v4.1	16 / 0	76	79.52	70	8.41	parallel waves of 4; plateau halt
v4.2 (aborted)	4 / 7	79	80.56	20	2.37	CLI auto-update broke the transport: 7 iterations poisoned, crash
v4.2	12 / 0	80	80.75	61	7.75	consultation-1 fixes; plateau halt
v4.3	14 / 0	81	81.48	68	9.09	cycle 1: shared on-policy data, ensembles; plateau halt
v4.4	12 / 0	81	81.38	58	7.59	cycle 2: per-experiment DAgger; plateau halt
v4.5	12 / 0	82	82.28	57	7.42	cycle 3: reference recipe + early stop; plateau halt
Total	83				46.00	plus two consultations (3.04 + 3.38) and one smoke test

Scores of v4.0 and v4.1 are on the first evaluation design (50 scenarios) and are not comparable with v4.2 onward (200 scenarios, 50 turns of four seeds each). On the later design the v4.1 best scores 75.94.

4. The story, version by version

v4.0: first working factory

Sequential, one iteration at a time, about six minutes per iteration. A one-iteration live smoke test passed every check. Two simulator problems were fixed before the first real run: the teacher's labels used the true future (unlearnable), and a toy model already scored about 90 because the score was anchored on a weak teacher. The clairvoyant anchor replaced it.

v4.1: parallel waves

An asynchronous swarm was rejected because results merged in completion order would make state, escalation and stopping depend on thread timing. Synchronous waves were chosen instead: one Meta call proposes K coordinates, K pipelines run in parallel, and Python merges results in iteration order at a barrier. Four iterations took under six minutes, about four times faster. The simulator's wall-clock training budget was replaced with counted budgets so scores do not depend on machine load. The run halted at 16 iterations on the plateau rule.

First consultation (Opus 5.5, xhigh effort)

Diagnosis: the student already beat its teacher, so every dimension the Meta explored only tuned how well a capped teacher was copied; the Judge's rubric pushed the Meta toward boosting rare actions, which cost about seven points; and the stopping test could not resolve two-point differences against about one point of evaluation noise.

v4.2: better training signal

Added a stronger two-step teacher (v2), per-action teacher values with soft targets, a return-based fine-tuning hook, Python-computed analytics for the Meta, a Judge anchored to within three points of the simulator score, waves of two, and evaluation on 50 turns of four seeds. Teacher v2 with soft targets was worth about four points. Best: 80.75.

Second consultation and the plateau loop

Diagnosis: the imitation signal was used up again, 40 percent of slots were wasted on functionally identical models or on a silently broken feature function, and fine-tuning could not detect gains that small. From here every candidate change was tested offline first, on development scenarios the factory never scores on.

Cycle	Lever	Unbiased gain over the v4.2 best	What happened
1 (v4.3)	On-policy data from a fixed reference student; seed ensembles; duplicate detection with Meta refill; validity gates	+0.1 sim (none)	The offline gain came from states visited by the learner itself. A shared reference student's states do not help.
2 (v4.4)	Per-experiment DAgger: each model's own rollouts are labelled by the teacher and added to training	+0.35 sim (not significant)	The lever was validated at about +1.3 sim, but the search never combined it with the best training settings.
3 (v4.5)	Early stopping; the strongest measured recipe documented in ai_docs	+1.1 to +1.6 sim (real)	The first wave reproduced the reference recipe exactly. Confirmed on three independent scenario sets.

5. Offline probes (no language model involved)

5.1 Training-signal probes (280 development scenarios, 3 seeds per arm, paired against teacher v2 + soft targets)

Probe	Gain in return (+- standard error)	Gain in sim_score	Verdict
DAgger: learner's own states, labelled by teacher v2	+0.34 +- 0.10	+1.5	adopted
DAgger + temperature 0.5 + 3-seed ensemble	+0.53 +- 0.13	+2.3	strongest combination
soft-target temperature 0.5 instead of 1.0	+0.13 +- 0.08	+0.6	small gain
5-seed ensemble	+0.17 +- 0.13	+0.8	small gain
teacher that uses the student as rollout policy	+0.24 +- 0.15	+1.1	rejected (hurts with DAgger)
advantage-regression target	+0.05 +- 0.10	+0.2	null
teacher v2 with 32 imagined futures	+0.05 +- 0.09	+0.2	null
hard labels instead of soft targets	-0.25 +- 0.11	-1.1	worse

Probe	Gain in return (+- standard error)	Gain in sim_score	Verdict
one-step teacher v1	-0.65 +- 0.12	-2.9	worse
teacher that plans to the end of the episode	-0.92 +- 0.15	-4.0	much worse

Retraining one recipe with a different random seed moves the score by about 0.5 points on its own, which is why ensembles became the default.

5.2 Does the gain survive the real simulator protocol?

Variant (paired against the v4.2 best, two development sets pooled)	Gain in return
per-experiment DAgger, 1 seed	about +0.10
per-experiment DAgger + 3 seeds + temperature 0.5	about +0.30 +- 0.13 (about +1.3 sim)
same plus early stopping	about +0.27
early stopping alone	+0.07
shared on-policy data (cycle 1) + 3 seeds + temperature 0.5	+0.04

5.3 Grokking probe

Question: is there a late jump in generalisation that the training budget cuts off? Two recipes were trained to 100,000 optimizer steps (the budget allows about 1,000 to 3,000) at weight decay 0, 0.01 and 0.1. In all six runs return peaked within 500 to 3,000 steps and then declined (by 0.01 to 1.14 return); validation regret rose and the weight norm kept growing. No grokking: longer training is ordinary over-fitting to near-tie teacher targets. Weight decay above 0.1 was not tested.

5.4 Information ceiling

Student inputs	sim_score (development set)	Gain in return
observations only	79.18	-
+ scenario family and true parameters	78.68	-0.11 +- 0.14
+ true hidden state (salience, fact integrity, true redundancy and staleness)	81.44	+0.51 +- 0.12
+ both	80.20	+0.23 +- 0.13

Even perfect knowledge of the hidden state adds only about 2.3 points. With the ensemble and DAgger gains on top, the practical ceiling of this approach is about 83. The remaining distance to 100 is mostly the unpredictable future and the quality of the teacher, not missing observations.

6. Hidden confirmation set

Selecting the best model on the 200 evaluation scenarios inflates its score (winner's curse). The specification's rule is unchanged: best_model.py is the highest judge score. In addition, when the factory stops, every model statistically tied with the selected best (at most six) is retrained and scored on 200 hidden scenarios that are never used for selection.

Iteration (v4.5)	Judge	Evaluation sim_score	Hidden sim_score	Note
3	82	82.28	81.16	selected best (spec)
5	82	81.95	80.99	
6	82	81.89	80.40	
10	82	81.50	81.37	confirmed best
2	81	81.39	80.91	
11	79	81.16	81.21	

Iteration (v4.5)	Judge	Evaluation sim_score	Hidden sim_score	Note
v4.2 best (reference)	80	80.75	79.36	same effect one run earlier

All six v4.5 models are tied within noise at about 81 on the hidden set. Against the v4.2 best, iteration 0 (the reference recipe) gains +0.25, +0.44 and +0.40 return on three independent sets, and iteration 3 gains +0.08, +0.20 and +0.43.

7. Incidents and what was fixed

Incident	Cause	Fix
Seven iterations poisoned in one second, then a crash	The Claude Code auto-updater replaced claude.exe mid-run; the launch error was treated as a Meta failure	A launch error is now an infrastructure fault: back off, re-resolve the binary, then pause with resumable state. A circuit breaker pauses after two fully poisoned waves.
Run killed mid-wave	System memory critically low (browser using about 5 GB)	Resumed from the wave snapshot after memory was freed; a memory alert now runs alongside each run.
Two DAgger iterations failed a time guard	A 30-second guard covered rollouts and labelling as well as training	The guard now bounds each training fit separately; the 60-second kill still bounds the whole simulation.
Analytics one wave stale	Computed before the wave's iterations were closed	Computed after the wave closes. Found by a test.
Audit flagged a hard-coded state space (false alarm, twice)	Meta dimensions named like simulator configuration keys (finetune, data, ensemble_seeds)	The audit now derives its exemption list from the simulator's own key list.
A probe showed a catastrophic result	Label cache keyed by episode count, so two datasets shared labels	Cache keyed by episode seeds; probe re-run.
Meta descriptions rejected repeatedly	400-character limit on dimension descriptions	Raised to 800; the escalation ladder recovers the rest.

8. Using the MLP

The exported package is in workspace/deploy/: policy_runtime.py (loader, PyTorch only), policy_model.py (features and network), intervention_policy.pt (weights of the three ensemble members and the input normalisation), policy_card.json and README.md. The exported runtime reproduces the simulator's returns on all 50 evaluation turns.

```
from policy_runtime import InterventionPolicy

policy = InterventionPolicy.load("workspace/deploy")
history = []
for step in agent_steps():
    history.append(observe_context(agent, step))    # 21-field dict per step
    action = policy.decide(history)                # "NOOP" | "COMPACT" | ...
    apply_intervention(agent, action)              # your harness performs it
```

What an intervention is

An intervention is an operation the harness performs directly on the agent's context between steps. It is not advice to the agent and the agent is not asked to act. The policy was trained assuming the action happens immediately.

Action	What the harness does	Form
NOOP	nothing	-
COMPACT	summarise older messages and replace them with the summary	context rewrite
PRUNE_TOOLS	delete or blank old tool outputs, keep the latest	context edit
REINJECT_INSTRUCTIONS	append the task instructions again as a system-style reminder	inserted message

Action	What the harness does	Form
CHECKPOINT_RESET	start a fresh context seeded with a checkpoint summary	session swap
RETRIEVE_MEMORY	query memory or notes and insert the retrieved facts	inserted content

Caveats

- The policy was trained and tested only in the simulator. Real agents will differ; expect a sim-to-real gap.
- It saw a 128k-token window and 40-step sessions. Very different scales need rescaling or retraining.
- Start in shadow mode: log recommendations next to what actually happened before letting the harness act.
- For production, record real sessions in the 21-field schema and reuse the factory's training pipeline.

9. What has and has not been explored

Across the runs the Meta-Agent created 45 dimension names covering about 20 distinct ideas: optimiser settings, targets and teachers, training data (behaviour, shared on-policy, per-experiment DAgger), ensembles, belief-state and history features, a recurrent history encoder, decision calibration, fine-tuning and early stopping.

- **Worked:** teacher v2 with soft targets (about +4 sim); per-experiment DAgger with a 3-seed ensemble and temperature 0.5 (about +1.3).
- **Within noise:** optimiser settings, feature variants, history encoders, most logit-bias calibrations, early stopping.
- **Harmful:** class weighting and rare-action boosting, very long training, planning the teacher to the end of the episode.
- **Not explored:** attention or mixture-of-experts architectures, auxiliary heads predicting hidden variables, multi-round DAgger, search-based or learned-value teachers, reinforcement learning with a real budget, larger training sets.

10. Verification status

- 52 automated tests pass: router rules and audit, stopping rules, isolation guard, state-space validation, code gate, simulator determinism, DAgger, early stopping, fingerprinting, and end-to-end runs with a mock LLM (retries, poisoning, Fable violation, timeout, resume, circuit breaker, duplicate refill, hidden confirmation).
- Latest run (v4.5): 12 of 12 iterations pass every per-iteration check; all nine whole-run checks pass.
- Exported policy: all 50 evaluation turns identical to the simulator; the single-decision and batched interfaces agree on every sampled decision.

11. Where everything is

Path	Contents
specs/optimizer/master_spec.md	The original specification
specs/optimizer/implementation_plan.md	The plan and the full engineering log: sections 0-8 design, 9-12 each version and cycle, 13 confirmation set, 14 information ceiling
specs/optimizer/consultation_opus55_xhigh.md, consultation2_opus55_xhigh.md	Both Opus 5.5 xhigh diagnoses (with their prompts)
specs/optimizer/probe_results.json, grokking_results.json, privileged_results.json	Raw probe measurements
specs/optimizer/project_report.pdf	This report
workspace/research_paper.md	The paper written by the factory's Documenting Agent (v4.5 run)
workspace/best_model.py, best_model_confirmed.py, confirmation.json	Selected best (spec rule), hidden-set confirmed best, confirmation table

Path	Contents
workspace/deploy/	Deployable policy package and integration guide
workspace/factory_report.json, factory.log	Definition-of-Done report and run log for the latest run
experiments/exp_000 ... exp_011	Latest run: model, config, simulation log, judge verdict, router log, prompts, isolation audit per iteration
experiments/_archived/	All earlier runs with their workspaces
factory/	Harness, simulator, prompts, probes (factory/probes/), tests (factory/tests/)
ai_docs/	Simulator contract, concepts, feature catalog and reference recipe given to the Worker

To reproduce: `.venv/Scripts/python.exe factory/run_factory.py --max-iterations 50 --parallel 2 --fresh`. To re-export a model: `.venv/Scripts/python.exe -m factory.export_policy [iteration]`.